

Actuá como un agente experto en ESP32, MicroPython, BLE, WiFi, MQTT, uasyncio, arquitectura de datos y firmware robusto para IoT. Tu tarea es DISEÑAR e IMPLEMENTAR desde cero el firmware del sistema “Dosimat IoT”, respetando exactamente esta arquitectura funcional y de comunicación.

OBJETIVO GENERAL

Construir un firmware extremadamente estable, seguro y escalable, con:

- Comunicación BLE minimalista
- Comunicación WiFi + MQTT robusta
- Exclusión mutua BLE/WiFi (nunca simultáneos)
- Máquina de estados de red
- Arquitectura de datos basada en IDs cortos
- Firebase como SSOT (vía MQTT + Cloud Functions)
- Historial optimizado (buffer circular + append-only)
- Sincronización móvil <> Firebase <> ESP32
- Multi-equipo, multi-usuario y roles (propietario, técnico, super admin)

ARQUITECTURA MULTI-EQUIPO Y MULTI-USUARIO

El ESP32 NO maneja usuarios.

Solo reporta:

- CHIPID
- estado
- telemetría compacta
- logs compactos

Los roles se manejan en Firebase:

usuarios/{UID}/perfil

usuarios/{UID}/equipos_asignados/{CHIPID}: true

equipos/{CHIPID}/

estado/actual

config/actual

programas/actual

perfiles_dosis/actual

logs/{logId}

propietarios/{UID}

roles/

super_admin/{UID}: true

tecnicos/{UID}: true

ARQUITECTURA DE COMUNICACIÓN

BLE minimalista

El ESP32 transmite SOLO:

- IDs cortos (PR1, P1, etc.)
- estado corto (IDLE, FILTRO, DOSIS, PAUSA, ANTI, RESET)
- tiempos restantes
- comandos simples

Nunca transmite:

- nombres largos
- configuraciones completas
- perfiles completos

WiFi + MQTT robusto

- Conexión no bloqueante
- Reconexión automática
- Timeout TCP reducido
- Loop MQTT no bloqueante
- Publicación compacta

Exclusión mutua BLE/WiFi

- Si no hay WiFi → BLE ON, WiFi OFF

- Si se configura WiFi → BLE OFF, WiFi ON
- Si WiFi falla → BLE ON, WiFi OFF
- Si WiFi reconecta → BLE OFF, WiFi ON

MÁQUINA DE ESTADOS DE RED

Estados:

- INIT
- BLE_ONLY
- WIFI_CONNECTING
- WIFI_ONLINE
- FALLBACK_BLE

ARQUITECTURA DE DATOS

IDs cortos en ESP32

PR1, PR2, P1, P2, etc.

Nombres largos en Firebase

La PWA traduce IDs cortos a nombres largos.

Configuración mínima local

El ESP32 guarda solo:

- config_version
- tespera_seg
- tdosis_seg
- ajuste_baja
- temporada_alta_inicio
- temporada_alta_fin

Historial optimizado

- Buffer circular en RAM
- Archivo append-only rotado por tamaño

- Subida a Firebase solo si hay WiFi

LÓGICA FUNCIONAL DEL EQUIPO (ESTADOS LARGOS)

El ESP32 reporta a la PWA/Firestore los estados descriptivos:

- En_espera
- Filtrando
- Dosificando
- Pausado
- Antiatasco
- Reinicio

Internamente usa nombres cortos:

- IDLE
- FILTRO
- DOSIS
- PAUSA
- ANTI
- RESET

CONTROL DEL LED (FIRMWARE)

El ESP32 debe controlar un LED físico que comunica el estado del equipo al usuario.

El LED sigue patrones específicos según el estado del equipo y el modo de conexión.

El firmware debe implementar estos patrones usando uasyncio:

```
LED_PATRONES = {  
    'En_espera_wifi':    [(1,200),(0,4000)],  
    'En_espera_ble':    [(1,200),(0,2000)],  
    'inactivo_refuerzo': [(1,200),(0,200),(1,200),(0,4000)],  
    'dosificando':      [(1,1000),(0,1000)],  
    'dosificando_refuerzo': [(1,4000),(0,200)],  
    'solo_bomba':       [(1,500),(0,500)],
```

```
'mantenimiento':    [(1,200),(0,200)]  
}
```

Reglas:

- El LED se controla localmente, sin depender de Firestore.
- El LED se actualiza automáticamente cuando cambia el estado del equipo.
- El LED debe funcionar tanto en modo BLE_ONLY como en WIFI_ONLINE.
- El LED debe usar nombres cortos internos (IDLE, FILTRO, DOSIS, PAUSA, ANTI, RESET).
- El patrón “En_espera_wifi” se usa cuando el estado es En_espera y WiFi está activo.
- El patrón “En_espera_ble” se usa cuando el estado es En_espera y BLE está activo.
- Si refuerzo está activo, usar el patrón correspondiente.
- El LED debe ejecutarse en loop con uasyncio, sin bloquear el firmware.

TAREAS DEL AGENTE

1. Diseñar arquitectura completa del firmware.
2. Definir módulos y funciones.
3. Definir estructura mínima local.
4. Definir estructura completa en Firebase.
5. Diseñar máquina de estados BLE/WiFi.
6. Diseñar BLE minimalista.
7. Diseñar WiFi robusto.
8. Diseñar MQTT no bloqueante.
9. Diseñar sincronización con Firebase.
10. Diseñar historial optimizado.
11. Generar código MicroPython.
12. Diseñar soporte multi-equipo/multi-usuario.
13. Proponer plan de migración futura.

ENTREGABLES

- Arquitectura completa
- Módulos y funciones

- Máquina de estados
- Diagrama de flujo
- Diseño BLE
- Diseño WiFi/MQTT
- Sincronización Firebase
- Código MicroPython
- Diagnóstico de riesgos